

Super Mario Game — 2-Member Task Division (Vertical Slices)

Project: 110-feature Super Mario Bros (C++17, SFML 3.0.2) **Reference:** [TASKS.md](#), [SPEC.md](#)
Phase 0 (Environment Setup) is already complete and shared.

Division Philosophy

Instead of splitting by horizontal layer (one person does all systems, the other does all UI), we split into **two vertical domain slices**. Each member builds **both engine/systems code AND graphics/UI/presentation code** for their domain:

Domain Slice	Member	Systems Work	Presentation Work
 Player & World	Member A	Core engine, Physics, Player entities, Items, Player states, TileMap, Levels, Camera, Save/Load	Parallax, Water/Lava VFX, Screen transitions, MenuState, WorldMapState, PlayingState, OptionsState, 2P mode
 Enemies & Interaction	Member B	InputManager/Commands, SoundManager, Enemies, AI strategies, Blocks, EntityFactory, Object Pool, Replay, Debug Console	SpriteSheet, Animation system, HUD, Minimap, Particles, Death/Invincibility FX, Audio wiring, CharSelect, Pause, GameOver, Victory, Statistics, Achievements, Boss fights, Polish

[!IMPORTANT] **Both members write C++ systems code AND visual/UI code.** Neither member is pigeonholed into one layer. Both gain full-stack experience across the entire architecture.

Member A — Player & World Domain

Systems & Engine Work

Phase 1 — Core Infrastructure (partial)

- ☐ 1.1 Constants & Utilities — `Constants.hpp` , `MathUtils.hpp/.cpp`
- ☐ 1.2 Resource Manager (Singleton) — `ResourceManager.hpp/.cpp`
- ☐ 1.4 Event Bus (Observer Pattern) — `EventBus.hpp/.cpp` (15+ event types)
- ☐ 1.6 Game State Manager (State Pattern) — `IGameState.hpp` , `GameStateManager.hpp/.cpp`
- ☐ 1.7 Game Class (Singleton) — `Game.hpp/.cpp` , fixed-timestep loop, ImGui integration
- ☐ Commit: `feat: implement core infrastructure (Game, GSM, ResourceManager, EventBus)`

Phase 2 — Physics Engine (full)

- ☐ 2.1 AABB — `AABB.hpp/.cpp`
- ☐ 2.2 Spatial Hash — `SpatialHash.hpp/.cpp`
- ☐ 2.3 Collision Detector — `CollisionDetector.hpp/.cpp`
- ☐ 2.4 Collision Resolver — `CollisionResolver.hpp/.cpp` (stomp/damage/bounce/collect, surface types)
- ☐ 2.5 Physics Engine — `PhysicsEngine.hpp/.cpp` (gravity, velocity, coyote time, jump buffer, water/ice/conveyor, ImGui debug)
- ☐ Commit: `feat: implement physics engine with collision pipeline`

Phase 3 — Player Entities & Items (partial)

- ☐ 3.1 Entity base class (abstract)
- ☐ 3.2 Character base class (abstract)
- ☐ 3.3 Player base class — IPlayerState pattern, 5 base states (Small, Super, Fire, Cape, Mini), 2 decorators (Star, Mega)
- ☐

3.4 Mario — standard physics, fireball shooting, animation state tracking

- ☐ 3.5 Luigi — modified physics, double jump
- ☐ 3.6 Toad & Peach (unlockable characters)
- ☐ 3.10 Item base + original items — Mushroom, FireFlower, Coin, Star, OneUpMushroom
- ☐ 3.11 New items (v2.0) — CapeFeather, MegaMushroom, MiniMushroom, POWBlock, PSwitch, Trampoline, StarCoin
- ☐ Commit: `feat: implement player characters, player states, and all items`

Phase 4 — Tilemap & Levels (full)

- ☐ 4.1 TileMap — 2D grid, tile properties, surface types, P-Switch swap, water zones
- ☐ 4.2 Level Loader — JSON parsing per SPEC §9.6, entity spawning via Factory
- ☐ 4.4 Level files — `level_1.json`, `level_2.json`, `level_3.json`, `bonus_1.json`, `entities.json`
- ☐ 4.5 Integration test — PlayingState loads Level 1, renders tilemap, spawns Mario
- ☐ Commit: `feat: implement tilemap, level loader, and level JSON files`

Phase 8 — Save/Load & Persistence (full)

- ☐ 8.1 Serializer — save/load slots using SPEC §12.2 expanded schema
- ☐ 8.2 Achievement Manager — subscribes to EventBus, checks conditions
- ☐ 8.3 Statistics Tracker — subscribes to EventBus, accumulates stats
- ☐ 8.4 Save/Load Integration — auto-save, manual save, settings persistence
- ☐ Commit: `feat: implement save/load, achievements, and statistics`

Presentation & Gameplay Work

Phase 4 — Camera (from Phase 4)

- ☐ 4.3 Camera — smooth follow, lookahead, bounds clamping, multi-mode scroll, screen shake

- ☐
Commit: `feat: implement camera with multi-mode scrolling`

Phase 5 — World Visuals (partial)

- ☐
5.5 Parallax Background — multi-layer scrolling
- ☐
5.7 Screen Transitions & Screen Shake — fade-in/out, light/medium/heavy shake
- ☐
5.10 Water & Lava Animation — sine-wave water surface, bubble/ember particles
- ☐
Commit: `feat: implement parallax, screen transitions, water/lava VFX`

Phase 7 — Player-Facing Game States (partial)

- ☐
7.1 Animated Menu State — animated background, running Mario, spinning coins, attract mode after 30s idle
- ☐
7.3 World Map State — overhead map with level nodes, animated paths, star coin display, sequential level unlock
- ☐
7.4 Playing State (Full Implementation) — integrate level load, player spawn, physics, camera, HUD, minimap, all collision callbacks, swimming/climbing/wall sliding/ground pounding, P-Switch timer, combo system, star coin tracking, ImGui dev panel
- ☐
7.8 Options State & High Scores — volume sliders, difficulty selection, controls rebinding, colorblind mode toggle, high score persistence
- ☐
Commit: `feat: implement Menu, WorldMap, Playing, Options states`

Phase 11 — Two-Player & Meta-Game (partial)

- ☐
11.1 Two-Player Versus Mode — Player 2 keyboard mappings, shared camera following leading player, versus rules
 - ☐
11.3 Meta-Game — New Game+ (mirrored levels, faster enemies), Daily Challenge (date-seeded procedural level), unlockable character conditions
 - ☐
Commit: `feat: implement 2P mode, NG+, daily challenge, unlockables`
-

Member A Summary Table

Category	Files/ Tasks	Examples
 Systems	~50 files	Game, Physics, Player, Items, TileMap, LevelLoader, Serializer, AchievementManager
 Presentation	~25 files	Camera, Parallax, Water/Lava VFX, MenuState, WorldMapState, PlayingState, OptionsState, Screen transitions
Design Patterns	8 patterns	Factory (shared), Singleton, State (IPlayerState + IGameState), Observer (EventBus), Decorator (Star/Mega), Memento (via Save)

Member B — Enemies & Interaction Domain

Systems & Engine Work

Phase 1 — Input & Sound (partial)

- ☐ **1.3** Sound Manager (Singleton) — `SoundManager.hpp/.cpp` (play SFX, stream BGM, volume controls, footstep support)
- ☐ **1.5** Input Manager (Command Pattern) — `InputManager.hpp/.cpp`, 8+ ICommand classes (Jump, Move, Fire, Crouch, GroundPound, WallJump, Run, Debug), Player 1/2 key mappings, key rebinding
- ☐ **1.8** Placeholder States — barebones `MenuState` and `PlayingState` stubs (for early testing)
- ☐ Commit: `feat: implement InputManager with commands and SoundManager`

Phase 3 — Enemies, Blocks & Factory (partial)

- ☐ **3.7** Enemy base class + IMovementStrategy interface + 8 concrete strategies (Patrol, Chase, Fly, TimerEmergence, Linear, HammerThrow, TetheredChase, ProximityTrigger)
- ☐ **3.8** Original enemies — Goomba, KoopaTroopa, KoopaParatroopa, Boo (with variants)
- ☐ **3.9** New enemies (v2.0) — PiranhaPlant, BulletBill, HammerBro, Thwomp, ChainChomp, Lakitu, Spiny
- ☐

3.12 Block base + original blocks — BrickBlock, QuestionBlock, Pipe, Flagpole

- ☐ 3.13 New blocks (v2.0) — HiddenBlock, MovingPlatform, FallingPlatform, IceBlock, ConveyorBelt
- ☐ 3.14 Entity Factory — `EntityFactory.hpp/.cpp`, 25+ entity types, config-driven
- ☐ Commit: `feat: implement enemies, strategies, blocks, and EntityFactory`

Phase 10 — Advanced Systems (full)

- ☐ 10.1 Object Pool — `ObjectPool<T>` template, wire to fireballs/particles/BulletBills
 - ☐ 10.2 Config-Driven Entities — `entities.json` parsing, update EntityFactory
 - ☐ 10.3 Replay System — `ReplayRecorder.hpp/.cpp`, record/playback input commands
 - ☐ 10.4 Debug Console — `DebugConsole.hpp/.cpp`, toggle with ~, text→command, autocomplete
 - ☐ Commit: `feat: implement ObjectPool, config entities, replay, debug console`
-

Presentation & Gameplay Work

Phase 5 — Visual Systems (partial)

- ☐ 5.1 SpriteSheet Handler — `SpriteSheet.hpp/.cpp`
- ☐ 5.2 Animation System — `Animation.hpp/.cpp`, `AnimationManager.hpp/.cpp` (11+ animation states)
- ☐ 5.3 HUD — Score, Coins, World, Time, Lives, Combo counter, P-Switch timer bar, Boss health bar, Star coin indicators, floating score text
- ☐ 5.4 Minimap — 200×40 pixel overview, toggleable with M key
- ☐ 5.6 Particle System — `ParticleSystem.hpp/.cpp`, object-pooled, 8+ particle types (BrickBreak, CoinSparkle, DeathPoof, Stomp, Combo, WallDust, WaterBubble, LavaEmber)
- ☐ 5.8 Entity Death Animations — Goomba squish, enemy flip, Star kill launch, player death
- ☐ 5.9 Invincibility Visual FX — Star rainbow cycling + sparkle trail, hit sprite flashing
- ☐ 5.11 Source & Integrate Assets — download spritesheets, wire to all entities

- ☐

Commit: `feat: implement animation system, HUD, minimap, particles, VFX`

Phase 6 — Audio (full)

- ☐

6.1 Source Audio Assets — find 17+ SFX, 10+ BGM tracks

- ☐

6.2 Wire Sound Events — subscribe SoundManager to EventBus (15+ events), footstep sounds, combo SFX escalation, dynamic music layers

- ☐

6.3 Volume Controls — sliders in Options, persist to config.json

- ☐

Commit: `feat: wire all audio events with dynamic music`

Phase 7 — Interaction Game States (partial)

- ☐

7.2 Character Select State — Mario/Luigi side-by-side, locked/unlocked Toad/Peach display

- ☐

7.5 Pause State — transparent overlay with Resume, Restart, Save, World Map, Menu, Quit

- ☐

7.6 Game Over State — death counter, retry screen

- ☐

7.7 Victory State — score calculations, star coin summary, level transition

- ☐

7.9 Statistics State — total enemies, coins, deaths, time, combos

- ☐

7.10 Achievements Display — list screen from Main Menu, toast notification system (top-right slide-in)

- ☐

Commit: `feat: implement CharSelect, Pause, GameOver, Victory, Stats, Achievements`

Phase 9 — Enemy AI & Bosses (full)

- ☐

9.1 AI Behavior Tuning — tune all 13 enemy types + 3 variants, Lakitu spawner, Thwomp state machine, ChainChomp tether, ImGui AI debug overlay

- ☐

9.2 Boom Boom Mid-Boss — 3-phase boss for Level 2, charge→spin→recover, boss arena

- ☐

9.3 Bowser Boss — Phase 1 (walk + fire), Phase 2 (jump + faster fire), health bar, boss arena

- ☐

9.4 Difficulty Scaling — Easy/Normal/Hard DifficultyStrategy, entity distribution balancing

- ☐

Commit: `feat: tune AI, implement bosses, add difficulty modes`

Phase 11 — Polish & Accessibility (partial)

- ☐ **11.2** Edge Cases & Bug Fixes — respawn at checkpoint, flashing invincibility, shell chains, time-out death, 100-coin 1-UP, pipe transitions
- ☐ **11.4** Accessibility — colorblind mode (palette swap), audio navigation cues
- ☐ **11.5** Final Testing — full walkthrough all levels, all power-ups/enemies/blocks, save/load, 2P, difficulty, 60fps verification
- ☐ Commit: `feat: fix edge cases, add accessibility, final testing`

Member B Summary Table

Category	Files/ Tasks	Examples
 Systems	~45 files	InputManager, 8 Commands, SoundManager, Enemy base, 8 Strategies, 13 Enemies, Block base, 9 Blocks, EntityFactory, ObjectPool, Replay, DebugConsole
 Presentation	~30 files	SpriteSheet, Animation, HUD, Minimap, ParticleSystem, Death/Invincibility FX, Audio wiring, CharSelectState, PauseState, GameOverState, VictoryState, StatisticsState, AchievementsDisplay, Boss fights
Design Patterns	7 patterns	Factory (EntityFactory), Strategy (8 AI strategies), Command (8 input commands), Template Method (strategy hooks), Object Pool, State (enemy/block lifecycles)

Per-Member Balance: Systems vs. Presentation

	Member A	Member B
Systems work	~50 files (~65%)	~45 files (~60%)
Presentation work	~25 files (~35%)	~30 files (~40%)
Total files	~75	~75
Design patterns	8	7
Est. duration	~6–7 weeks	~6–7 weeks
Key systems	Physics, Player states, Level pipeline, Save/Load	Input/Commands, Enemy AI, Blocks, EntityFactory, Advanced Systems
Key presentation	Camera, Menu, WorldMap, Playing, Options, Parallax, Water/Lava	Animation, HUD, Particles, Audio, CharSelect, Pause, GameOver, Victory, Bosses

Dependency Timeline

```
gantt
    title Super Mario Game – 2-Member Vertical Slices
    dateFormat YYYY-MM-DD
    axisFormat %b %d

    section Member A (Player & World)
    Ph1 Core Infrastructure      :a1, 2026-06-09, 5d
    Ph2 Physics Engine         :a2, after a1, 7d
    Ph3 Player & Items         :a3, after a1, 12d
    Ph4 Tilemap & Levels       :a4, after a2, 7d
    Ph5 World Visuals         :a5, after a4, 5d
    Ph7 Menu/WorldMap/Playing :a7, after a5, 10d
    Ph8 Save/Load              :a8, after a7, 5d
    Ph11 2P & Meta-game        :a11, after a8, 5d

    section Member B (Enemies & Interaction)
    Ph1 Input & Sound          :b1, 2026-06-09, 4d
    Ph3 Enemies/Blocks/Factory :b3, after b1, 14d
    Ph5 Animation/HUD/VFX     :b5, after b1, 10d
    Ph6 Audio                  :b6, after b5, 4d
    Ph7 CharSelect/Pause/etc   :b7, after b6, 8d
    Ph9 AI Tuning & Bosses     :b9, after b3, 10d
    Ph10 Advanced Systems      :b10, after b9, 7d
    Ph11 Polish & Testing       :b11, after b10, 5d

    section Integration Sync
    Sync 1 Core APIs ready     :milestone, after a1, 0d
    Sync 2 Entities compiled   :milestone, 2026-06-30, 0d
    Sync 3 Levels playable     :milestone, after a4, 0d
    Sync 4 Final integration    :milestone, 2026-07-28, 0d
```

Integration Sync Points

Sync	When	Member A provides	Member B provides
Sync 1	Week 1 end	<code>Game</code> , <code>GameManager</code> , <code>IGameState</code> , <code>EventBus</code> , <code>ResourceManager</code> APIs	<code>InputManager</code> , <code>SoundManager</code> , <code> ICommand</code> interface
Sync 2	Week 3 end	Entity/Character/Player base classes, Item classes	Enemy/Block classes, EntityFactory, AI strategies
Sync 3	Week 4 end	TileMap, LevelLoader, Camera, Level JSON files	Animation system, SpriteSheet, HUD (ready to wire)
Sync 4	Final week	Save/Load, 2P mode, Meta-game features	Object Pool, Replay, Bosses, polish, final testing

[!WARNING] **Key dependency:** Member B's EntityFactory (3.14) needs Member A's Entity/Character/Player/Item base classes and Member B's own Enemy/Block classes. Coordinate on Phase 3 — Member A should commit base classes first, then both work in parallel on their concrete entities.

File Ownership Map

Directory / File	Member A	Member B	Notes
Core/Game.* , Core/GameStateManager.* , Core/IGameState.*	✓	✗	A owns engine infrastructure
Core/ResourceManager.* , Core/EventBus.*	✓	✗	A owns resource and event systems
Core/InputManager.* , Core/SoundManager.*	✗	✓	B owns input and sound
Core/AchievementManager.* , Core/StatisticsTracker.*	✓	✗	A owns persistence-related managers
Core/MenuState.* , Core/WorldMapState.*	✓	✗	A owns player-facing flow states
Core/PlayingState.*	✓ primary	✓ contributes	Both contribute — A writes skeleton, B adds HUD/audio hooks
Core/OptionsState.*	✓	✗	A owns options/settings
Core/CharSelectState.* , Core/PauseState.*	✗	✓	B owns interaction states
Core/GameOverState.* , Core/VictoryState.*	✗	✓	B owns outcome states
Core/StatisticsState.* , Core/ReplayRecorder.*	✗	✓	B owns stats display and replay
Entities/Entity.* , Character.* , Player.*	✓	✗	A owns base hierarchy
Entities/Mario.* , Luigi.* , Toad.* , Peach.*	✓	✗	A owns player characters
Entities/IPlayerState.* , Small/Super/Fire/Cape/MiniState.*	✓	✗	A owns player state pattern
Entities/StarDecorator.* , MegaDecorator.*	✓	✗	A owns decorator pattern
Entities/Item.* + all 12 item subclasses	✓	✗	A owns all items

Directory / File	Member A	Member B	Notes
<code>Entities/Enemy.*</code> , <code>IMovementStrategy.*</code> + 8 strategies	✗	✓	B owns enemy hierarchy
All 13 enemy subclasses	✗	✓	B owns all enemies
<code>Entities/Block.*</code> + all 9 block subclasses	✗	✓	B owns all blocks
<code>Entities/EntityFactory.*</code>	✗	✓	B owns the factory (registers all types)
<code>Physics/*</code>	✓	✗	A owns all physics code
<code>Graphics/Camera.*</code>	✓	✗	A owns camera
<code>Graphics/Animation.*</code> , <code>AnimationManager.*</code> , <code>SpriteSheet.*</code>	✗	✓	B owns animation pipeline
<code>Graphics/HUD.*</code> , <code>Minimap.*</code>	✗	✓	B owns HUD and minimap
<code>Graphics/ParticleSystem.*</code>	✗	✓	B owns particles
<code>Utils/Constants.*</code> , <code>MathUtils.*</code>	✓	✗	A owns utilities
<code>Utils/TileMap.*</code> , <code>LevelLoader.*</code>	✓	✗	A owns level data
<code>Utils/Serializer.*</code>	✓	✗	A owns save/load
<code>Utils/ObjectPool.*</code> , <code>DebugConsole.*</code>	✗	✓	B owns advanced systems
<code>assets/levels/</code> , <code>assets/config/</code>	✓	✗	A designs level and config JSON
<code>assets/textures/</code> , <code>assets/sounds/</code>	✗	✓	B sources media assets

Shared Responsibilities

1. `PlayingState` (Phase 7.4) — The most complex file. Member A writes the update/physics loop skeleton. Member B adds HUD rendering, audio hooks, and animation wiring. **Both contribute.**
2. `CMakeLists.txt` — Both add source files. Coordinate each merge to avoid conflicts.

3. **Animation State Names** — Agree early on enum values (idle, walk, run, jump, crouch, slide, swim, climb, wall_slide, ground_pound, die) so Member A's entities and Member B's animation system use the same vocabulary.
4. **EventBus Events** — Member A defines the `EventType` enum and publishes events from entities/physics. Member B subscribes from HUD/Sound/Achievement. Both must agree on event data payloads.
-

Bonus Phase Ownership

Bonus	Owner	Rationale
A — Level Editor	Member A	ImGui-based, builds on TileMap/LevelLoader, level JSON pipeline
B — Time Rewind	Member A	Memento pattern, state snapshots, extends Save/Load systems
C — Shadow Mario	Member B	Input recording/replay, extends Replay system
D — Dynamic Lighting	Member B	GLSL shaders, visual effects, extends graphics pipeline
E — Procedural Generation	Member A	Chunk generation, extends level data pipeline